

Aula 8 | Tratamento de Erros



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Resolver o problema: como evitar que seu sistema exploda quando recebe dados inválidos, conexões falham ou operações inesperadas ocorrem. Entender e aplicar: try/catch/finally, o objeto Error, tipos de erro, throw e criação de erros próprios. Saber decidir quando capturar e continuar, e quando re-lançar e parar.

Problema gerador

Como evitar que seu sistema exploda?

Pergunta guia

Como capturar os erros que ocorrem durante a execução e decidir se o sistema deve continuar rodando ou parar com segurança? E como criar meus próprios erros para sinalizar problemas no código?

- Estudo de caso 1: Simulador de dados com validação e erros personalizados
- Expositiva: try / catch / finally e o objeto Error
- Expositiva: throw, tipos de erro e propagação
- Prático: Exercícios de fixação (tratamento de erros)

Conceitos

Tratamento de Erros

- `try` – bloco que tenta executar o código
- `catch(e)` – captura o erro e decide o que fazer
- `finally` – sempre executa, erro ou não (limpeza)
- `throw` – lança um erro intencional
- `Error`, `TypeError`, `RangeError`, `SyntaxError` – tipos nativos
- `e.name`, `e.message`, `e.stack` – informações do erro
- `Re-throw` – re-lançar erros inesperados para o chamador

try / catch em JavaScript

TRY/CATCH:

O try executa o bloco normalmente. Se qualquer linha lançar um erro, o JS pula imediatamente para o catch. O código APÓS o try/catch continua normalmente – o programa não para. Use try/catch para erros ESPERADOS e recuperáveis.

```
// Sintaxe básica
try {

    // código que pode lançar erro
    const n = Number(prompt('Digite um número:'));
    if (isNaN(n)) throw new Error('Não é número!');

    console.log('Raiz quadrada:', Math.sqrt(n));
} catch (e) {
    console.log('Erro capturado:', e.message);
}

// Sem try/catch: o programa PARA ao errar

// Com try/catch: o programa CONTINUA
```



O Objeto Error e Tipos de Erro

```
if (e instanceof SyntaxError) console.log('JSON inválido!');
```

O parâmetro `e` no catch é um objeto com name, message e stack.
O JS lança tipos específicos de erro para cada situação.

```
try {  
  null.propriedade;          // TypeError  
} catch (e) {  
  console.log(e.name);       // 'TypeError'  
  console.log(e.message);    // Cannot read ...  
}
```

```
// Tipos comuns de erro:
```

```
// TypeError   – operação em tipo errado (null.x)  
// RangeError  – valor fora do intervalo (new Array(-1))
```

```
// ReferenceError – variável não declarada
```

```
// SyntaxError – JSON.parse('{ruim}')
```

```
// Verificar tipo com instanceof:  
try { JSON.parse('{ruim}'); } catch (e) {
```



Estudo de Caso 1: Dados com Tratamento de Erros

```
function simularDados(n, lados) {
  if (!Number.isInteger(n) || n < 1)

    throw new RangeError('n deve ser inteiro ≥ 1');

  if (!Number.isInteger(lados) || lados < 2)

    throw new RangeError('lados deve ser ≥ 2');

  const r = [];
  for (let i = 0; i < n; i++)

    r.push(Math.floor(Math.random() * lados) + 1);
  return { rolagens: r, soma: r.reduce((s,v)⇒s+v,0) };
}

try {

  const r = simularDados(3, 6);
  console.log('Rolagens:', r.rolagens.join(', '));
  console.log('Soma:', r.soma);
} catch (e) {
  if (e instanceof RangeError)
    console.log('Parâmetro inválido:', e.message);
  else throw e; // re-throw inesperado
}
```

Validar ANTES de processar – lance RangeError para valores fora do esperado. O chamador decide o que fazer com o erro. throw encerra a função imediatamente – como um return, mas com erro. A função fica limpa: sem console.log, sem prompt – só lógica pura.



finally e Erros Personalizados

`finally` garante limpeza de recursos (arquivos, conexões, timers) independente do resultado. `throw` pode lançar qualquer valor, mas prefira `new Error(...)` para manter `name`, `message` e `stack`. O `finally` executa ANTES do `return` ou `throw` saírem do bloco.

`finally` sempre executa – erro ou não. Use para limpar recursos.

```
function processarDados(json) {  
  const conn = { fechar: () => console.log('Conexão fechada') };  
  try {  
  
    const dados = JSON.parse(json);  
  
    if (!dados.nome) throw new TypeError('nome obrigatório');  
  
    console.log('OK:', dados.nome);  
  } catch (e) {  
  
    console.log('Erro:', e.message);  
  } finally {  
  
    conn.fechar(); // SEMPRE executa  
  }  
  
}
```



Continuar ou Parar? Estratégias de Tratamento

Nem todo erro deve encerrar o programa – decida baseado no contexto.

- Continuar: erro esperado e recuperável (entrada inválida, JSON malformado)
- Parar (re-throw): erro inesperado que o chamador precisa saber
- Logar sempre: registre o erro antes de continuar ou re-lançar

Padrão recomendado:

```
function processarEntrada(valor) {  
  const n = Number(valor);  
  
  if (isNaN(n)) throw new TypeError('Entrada inválida');  
  
  return n;  
}  
  
try {  
  const n = processarEntrada(prompt('Número:'));  
} catch (e) {  
  if (e instanceof TypeError)  
    console.log('Aviso: tente de novo.');  else throw e; // re-throw inesperado  
}  
console.log('Resultado:', n * 2);
```

Exercícios de fixação

Grupo 1: “Calculadora segura”

Adicione tratamento de erros a uma calculadora de operações básicas.

```
ex: function calcular(a, op, b) { /* sua implementação */ }
```

1. Escreva `calcular(a, op, b)` que suporta `+`, `-`, `*`, `/`.
2. Lance `TypeError` se `a` ou `b` não forem números.
3. Lance `Error` se `op` não for um dos 4 operadores válidos.
4. Lance `RangeError` se tentar dividir por zero.
5. Use `try/catch` para chamar `calcular` e exibir o resultado ou o erro.
6. Use `instanceof` no `catch` para diferenciar os tipos de erro e dar mensagens distintas.
7. ****Desafio:**** adicione `finally` que exibe sempre: `"Operação concluída."` – erro ou não.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exercícios de fixação

Grupo 2: “Parser de dados”

Trate erros ao processar dados externos em JSON.

ex:

```
const entradas = ['{"nome":"Ana"}', '{RUIM}', '{"nome":123}'];
```

1. Para cada string: tente `JSON.parse` e exiba o resultado ou o tipo do erro.
2. Use `instanceof SyntaxError` para detectar JSON malformatado.
3. Lance `TypeError` se `nome` não for uma string.
4. Use `finally` para exibir sempre: `"Processado: [valor ou ERRO]"`.
5. Escreva `parseSafe(str)` que retorna `null` em vez de lançar erro.
6. Acumule os erros em um array e exiba todos ao final.
7. ****Desafio:**** diferencie erros de parse de erros de validação e exiba contagens separadas.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!

Exercícios de fixação

Grupo 3: “Sistema de cadastro”

Implemente cadastro de usuários com validação robusta e erros próprios.

ex:

```
const usuarios = [];  
function cadastrar(nome, idade, email) {  
  // sua implementação aqui  
}  
// cadastrar("Ana", 25, "ana@x.com") → ok
```

```
// cadastrar("", 25, "ana@x.com") → TypeError
```

1. Lance `TypeError` se nome estiver vazio ou não for string.
2. Lance `RangeError` se idade não for inteiro entre 0 e 120.
3. Lance `Error` se email não contiver `@`.
4. Use try/catch ao chamar `cadastrar` e exiba mensagens amigáveis ao usuário.
5. Use `finally` para exibir a lista atual de usuários após cada tentativa.
6. Acumule cadastros bem-sucedidos em `usuarios` e ignore os com erro.

7. ****Desafio:**** crie uma classe `ValidationError extends Error` com campo `campo`.

Correção

```
function calcular(a, op, b) {  
  if (typeof a !== 'number' || typeof b !== 'number')  
  
    throw new TypeError('a e b devem ser números');  
  if (!['+', '-', '*', '/'].includes(op))  
    throw new Error(`Operador inválido: ${op}`);  
  if (op === '/' && b === 0)  
    throw new RangeError('Divisão por zero');  
  const ops = { '+': a+b, '-': a-b, '*': a*b, '/': a/b };  
  
  return ops[op];  
  
}  
  
try {  
  console.log(calcular(10, '/', 0));  
} catch (e) {  
  if (e instanceof RangeError)  
    console.log('Erro de intervalo:', e.message);  
} finally {  
  console.log('Operação concluída.');}
```



Correção

```
const entradas = ['{"nome":"Ana"}', '{RUIM}', '{"nome":123}'];
```

```
const erros = [];
```

```
function parseSafe(str) {
```

```
  try { return JSON.parse(str); }
```

```
  catch (e) { return null; }
```

```
}
```

```
for (const str of entradas) {
```

```
  try {
```

```
    const obj = JSON.parse(str);
```

```
    if (typeof obj.nome === 'string')
```

```
      throw new TypeError('nome deve ser string');
```

```
    console.log('OK:', obj.nome);
```

```
  } catch (e) {
```

```
  } finally {
```

```
    console.log('Processado:', parseSafe(str) ?? 'ERRO');
```

```
  }
```

```
}
```



Correção

```
const usuarios = [];  
  
function cadastrar(nome, idade, email) {  
  
    if (typeof nome !== 'string' || nome.trim() === '')  
        throw new TypeError('nome inválido');  
  
    if (!Number.isInteger(idade) || idade < 0 || idade > 120)  
        throw new RangeError('idade deve ser 0-120');  
  
    if (!email.includes('@'))  
  
        throw new Error('email inválido');  
    usuarios.push({ nome, idade, email });  
  
}  
  
try {  
    cadastrar('Ana', 25, 'ana@exemplo.com');  
    cadastrar('', 25, 'x@x.com');  
  
} catch (e) {  
    console.log(`Erro [${e.name}]: ${e.message}`);  
    console.log('Usuários:', usuarios.length);  
}
```



Pesquisa Individual

Tratamento de Erros em JavaScript

Leituras recomendadas:

- MDN try/catch:
[developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/try ... catch](https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/try...catch)
- MDN Error:
developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Error

Perguntas de pesquisa (10–15 min)

- 1) O que acontece se lançar um erro dentro do ``catch``? E dentro do ``finally``?
(Dica: erros no `finally` substituem o erro original – cuidado!)
- 2) O que é "error swallowing"? Por que um `catch` vazio é considerado má prática?
- 3) Quando usar ``instanceof`` vs ``e.name === 'TypeError'``? Qual é mais robusto?
- 4) O que é a pilha de chamadas (call stack) e como ``e.stack`` ajuda a depurar?
- 5) Bônus: pesquise como criar erros customizados com ``class MinhaError extends Error``.



Próxima aula

Aula 9

Programação assíncrona: callbacks, Promises e async/await – lidar com operações que levam tempo (rede, arquivos).

Para chegar bem:

- Traga dúvidas sobre try/catch e os exercícios
- Complete os 3 exercícios de fixação
- Pesquise a diferença entre erro síncrono e assíncrono em JS



Obrigado